

基于 CAN 总线的汽车诊断协议 UDS (网络层 ISO15765)

上个月一个同事 Z 跳槽去了德赛西威，Z 之前是完全不懂诊断的 MCU 工程师，去德赛后做诊断开发，让我感觉到，汽车嵌入式行业，CAN 和诊断工程师还是比较稀缺的。之前我和 Z 共同负责一个项目，我负责 CAN 网络和诊断部分，经过 4 个多月的奋战，我一个人把汽车诊断 UDS 的系统搭建出来，自认为，完成度很高，代码质量也极好。他跳槽去德赛做诊断开发，我想多少有点受益于我开发的诊断代码，另外我也悉心指导他，讲解相关的知识，他确实也学到不少，即便是现在，他有问题也会打电话向我求助。

前两年的工作和学习，我了解到汽车 CAN 网络和诊断还是比较难以学习的，网上资料参差不齐，我花了很大功夫才把这部分掌握，所以考虑写几篇相关的文章，以帮助后来者。

网络层的国际标准是 ISO15765-2，该标准详细规定了协议的具体细节。CAN 总线是一帧 8 个字节，该协议可以使 CAN 总线高效的传输大约 8 个字节（up to 4095 bytes）的命令和数据。基于该标准文档，我开发出了一个独立性良好的协议栈，工作在上层诊断协议之下和下层 CAN 驱动之上，下面详解开发协议栈时需要实现的部分（基于 ISO 15765-2:2004（E））

4 Network layer overview

4.2 Services provided by network layer to higher layers

4.2 小节是描述网络层协议提供给上层的服务

(a) Communication services （通信服务）

有四个，其中第 1 个是发送消息的服务，我实现为一个外部函数，提供给上层调用，第 2，3，4 是上层获取协议栈发送和接收状态的服务，我按照回调函数的方式实现，于是变成了上层提供给网络层的接口。如果转成 C++ 代码，可以用虚函数来实现。

1) N_USData.request

是网络层提供给上层的发送消息的服务，5.2.1 小节对其有详细的描述，我只实现了两个参数，msg_buf 和 msg_dlc，发送时根据消息长度判断是单帧发送还是多帧发送，

```
extern void network_send_udsmg (uint8_t msg_buf[],uint16_t msg_dlc)
{
    if (msg_dlc==0|| msg_dlc> UDS_FF_DL_MAX) return;

    if (msg_dlc<= UDS_SF_DL_MAX)
    {
        send_singleframe (msg_buf, msg_dlc);
    }
}
```

```

    }
    else
    {
        nwl_st = NWL_XMIT;
        send_multipleframe (msg_buf, msg_dlc);
    }
}

```

2)N_USData_FF.indication

该服务用来通知上层，网络层收到了首帧，5.2.3 小节对其有详细的描述，我实现了一个参数 `msg_dlc`，该函数通过回调实现，具体细节在上层代码中，按下不表。

函数原型声明如下

```
typedef void (*ffindication_func) (uint16_t msg_dlc);
```

网络层接收到首帧后调用该服务。

3)N_USData.indication

该服务把接收到的完整消息传递给上层，5.2.4 小节对其有详细的描述，我实现了 3 个参数，`msg_buf`，`msg_dlc` 和 `n_result`，该函数通过回调实现，具体细节在上层代码中，按下不表。

函数原型声明如下

```
typedef void (*indication_func) (uint8_t msg_buf[], uint16_t msg_dlc, n_result_t n_result);
```

该函数调用较多：

- 1.接收到单帧，with `N_OK`
- 2.接收连续帧，如果 `sn` 错误，with `N_WRONG_SN`
- 3.接收连续帧，如果长度正确，with `N_OK`
- 4.网络层主循环中，如果 `CR` 定时器超时，with `N_TIMEOUT_Cr`
- 5.接收到首帧和单帧，如果网络层状态异常，with `N_UNEXP_PDU`

4)N_USData.confirm

该服务用来通知上层，消息发送已经完成，并返回成功与否，5.2.2 小节对其有详细的描述。我实现了 1 个参数 `n_result`，该函数通过回调实现。具体细节在上层代码中，按下不表。

函数原型声明如下

```
typedef void(*confirm_func)(n_result_t n_result);
```

该函数调用如下：

- 1.接受到流控帧，如果流状态 $\geq$ FS_RESERVED, with N_INVALID_FS
- 2.接收到流控帧，如果流状态 $==$ FS_OVERFLOW, with N_BUFFER_OVFLW
- 3.网络层主循环中，如果 BS 定时器超时，with N_TIMEOUT_Bs

b) Protocol parameter setting services （协议参数控制服务）

协议参数控制服务有两个，我没有实现，具体用处我还不明白，但是不影响实现协议栈功能。

6 Network layer protocol

第 6 节描述网络层协议内容

6.1-6.4 小节简要说明

当消息长度小于等于 6（扩展地址和混合地址）或者 7（普通地址）个字节时，是通过一个 N_PDU(数据单元)发送完成，叫做 SF（单帧）。

当消息长度较大时，是通过多个 N_PDUs(数据单元)发送完成，这种数据单元叫做 FF（首帧，第一个 N_PDU）和 CF(连续帧，后续的 N_PDUs)。

FF（首帧）包括前面 5 个（扩展地址和混合地址）或者 6 个（普通地址）字节的内容，1 个或者多个 CF（连续帧），每个 CF 包括后续的 6 个（扩展地址和混合地址）或者 7 个（普通地址）字节的内容，当然也可以少于 6 个或者 7 个字节。消息长度信息在 FF（首帧）中发送，所有的 CF（连续帧）在发送端被编号，以帮助接收者按顺序重组

消息。（最后一句话没什么卵用）

接收者通过 Flow control（流控帧）的机制，告知发送者自己有多大的接收能力。（其实就是每两个 FC 之间允许连续发送多少个 CF，每两个 CF 之间的时间不能过快）

Flow control 包含三个字段：

Flow status（FS），流状态，用来控制发送方接下来的行为，总共有三个定义，分别是 FC.CTS（继续发送），FC.WAIT(继续等待)，FC_OVFLW(缓存溢出，此时应该终止发送)。

Block Size（BS），每次收到流控帧之后，发送者最大可发送的连续帧的个数。

SeparationTimeMin (STmin)，两个连续帧之间的最小间隔。

综上所述，网络层共有 4 中数据单元类型：SF N_PDU，FF N_PDU， CF N_PDU， FC N_PDU。详细说明在 6.4 节，不再赘述。

Tale 2 是 N_PDU format (数据单元格式)，每个 N_PDU 由三个域组成。

Table 2 — N_PDU format

Address information	Protocol control information	Data field
N_AI	N_PCI	N_Data

在使用普通地址时，地址域仅由 CAN ID 组成，CAN 消息数据的第一个字节（或前两字节）为 N_PCI Bytes。N_PCI(Protocol control information)标识了一条消息的类型和附加信息。

6.5 Protocol control information specification

Table 3 描述各种类型的 N_PDU 的 N_PCI bytes 的定义。

Table 3 — Summary of N_PCI bytes

N_PDU name	N_PCI bytes			
	Byte #1		Byte #2	Byte #3
	Bits 7 – 4	Bits 3 – 0		
SingleFrame (SF)	N_PCIttype = 0	SF_DL	N/A	N/A
FirstFrame (FF)	N_PCIttype = 1	FF_DL		N/A
ConsecutiveFrame (CF)	N_PCIttype = 2	SN	N/A	N/A
FlowControl (FC)	N_PCIttype = 3	FS	BS	STmin

N_PCI byte 的第一个字节的高 4 位为 N_PCIttype，标识该 N_PDU(数据单元)的类型。

- 0，SF（单帧）
- 1，FF（首帧）
- 2，CF（连续帧）
- 3，FC（流控帧）

4-F，保留定义

我在程序中接收到一条诊断报文后，通过一条宏定义获取 N_PCIttype

```
#define NT_GET_PCI_TYPE(n_pci) (n_pci>>4)
```

```
pci_type = NT_GET_PCI_TYPE (frame_buf[0]);
```

然后根据 `pci_type` 进行不同的处理。

(1)单帧的情况下，`N_PCI byte` 第一个字节的低 4 位为 `SF_DL`(消息长度)，范围在 1-6(扩展地址和混合地址)或者 1-7(普通地址)之间，如果 `SF_DL` 错误，网络层应该忽略这条 `N_PDU`

(2)首帧的情况下，`N_PCI bytes` 第一个字节的低 4 位和第二个字节共同组成 `FF_DL`(消息长度)，范围在 8-FFF（扩展地址和混合地址）或者 7-FFF（普通地址）之间，如果 `FF_DL` 大于接收者的接收缓存，网络层应该丢弃这条消息，并且发送 `FC with FlowStatus = Overflow`

(3)连续帧情况下，`N_PCI byte` 第一个字节的低 4 位为 `SN`（SequenceNumber），

在每开始发送一段数据的时候 `SN` 必须从零开始，`FF`（首帧）没有 `SN` 字段，但应该被认为是 `SN = 0`，

`FF` 之后的第一个 `CF` 的 `SN` 应该为 1，

每发送一个新的 `CF`，`SN` 都应该增加 1，

`CF` 的值不应该受 `FC` 的影响，

当 `SN` 的值达到 15 的时候，下次发送的 `CF`，`SN` 应被重置为 0，

如果 `SN` 出错，网络层应该丢弃已接收到的消息，并且调用 `N_USData.indication` 服务，

with `N_WRONG_SN`

(4)流控帧情况下，

`N_PCI bytes` 第一个字节的低 4 位为 `FS`（Flow status），`FS` 有 4 个定义，

0，`CTS`，代表发送者可以正常发送

1，`WT`，代表发送者应该再等待下一个 `FC`，并且重启 `N_BS timer`

2，`OVFLW`，代表接收方缓存溢出，发送方收到此 `FS` 后，应该终止发送，调用 `N_USData.confirm` 服务，with `N_BUFFER_OVFLW`

3-F，`Reserved`

如果发送者收到的 `FS` 出错，网络层应该停止消息发送，并且调用 `N_USData.confirm` 服务，

with `N_INVALID_FS`

`N_PCI bytes` 的第 2 个字节为 `BS`（BlockSize），`BS` 代表发送方在收到下一个 `FC` 之前，应发送的 `CF` 的数量，只有最后一块数据，其 `CF` 的数量可以少于 `BS`，`BS` 的值分两个情况，

0，代表没有 `BS` 限制，发送方不必等待 `FC`，把所有的 `FC` 一次发送。

1-FF，代表发送方发送 `BS` 数量的 `CF` 后，需等待 `FC`，

N_PCI bytes 的第 3 个字节为 STmin，发送方收到 FC 后，应该把 STmin 保存下来，该值表明两个 CF 之间的最小间隔，STmin 的值定义如下图

Table 15 — Definition of STmin values

Hex value	Description
00 – 7F	SeparationTime (STmin) range: 0 ms – 127 ms The units of STmin in the range 00 hex – 7F hex are absolute milliseconds (ms).
80 – F0	Reserved This range of values is reserved by this part of ISO 15765.
F1 – F9	SeparationTime (STmin) range: 100 µs – 900 µs The units of STmin in the range F1 hex – F9 hex are even 100 microseconds (µs), where parameter value F1 hex represents 100 µs and parameter value F9 hex represents 900 µs.
FA – FF	Reserved This range of values is reserved by this part of ISO 15765.

如果发送方收到一个 FC，其 STmin 的值是 Reserved，则发送方应默认 STmin 为 7F（127ms）

STmin 参数体现在程序中就是一个定时器，发送完一帧 CF 后，应该立即启动 STmin timer

timer 超时之后才能发送下一个 CF，我的实现方式如下，nt_timer_run(TIMER_STmin) < 0 代表 STmin timer 超时。

```
if (nt_timer_run (TIMER_STmin) < 0)
{
    g_xcf_sn++;
    if (g_xcf_sn > 0x0f)
        g_xcf_sn = 0;
    OSMutexPend(UdsMutex,0,&err);
    send_len = send_consecutiveframe (&remain_buf[remain_pos],
remain_len, g_xcf_sn);
    remain_pos += send_len;
    remain_len -= send_len;

    if (remain_len > 0)
    {
        if (g_rfc_bs > 0)
        {
            g_xcf_bc++;
            if (g_xcf_bc < g_rfc_bs)
            {
                nt_timer_start (TIMER_STmin);
            }
        }
        else
        {
            /**
             * start N_Bs and wait for a fc.
```

```

        */
        g_wait_fc = TRUE;
        nt_timer_start (TIMER_N_BS);
    }
}
else
{
    nt_timer_start (TIMER_STmin);
}
else
{
    clear_network ();
}
OSMutexPost (UdsMutex);
}

```

6.6 Maximum number of FC.Wait frame transmissions (N_WFTmax)

6.6 节，最大 FC.Wait 次数，是本地（local）的参数，不包含在 FC 中，

指明接收方最大能连续发送多少个 FC.Wait，

这个上限参数应该在系统规划的时候由用户定义，

该参数只在接收消息的时候使用，

该参数如果为 0，则接收方应该禁用 FC.Wait，即不发送 FS = WT 的流控帧。

我实现的时候，默认了该参数为 0，实际是根本没定义该参数，也不使用 FC = WT 的流控帧，

6.7 Network layer timing

6.7.1 Timing parameters

Table 16 定义了网络层的时间参数值，以及各个时间参数的开始和结束点，这些体现通信性能的值，通信双方都应该满足，每个程序都可以定义具体的值，但是要在 Table 16 的范围内。（实际上，车厂会给一个文档，叫做诊断规范，会规定这些参数的值）

通常，将超时值定义为高于性能要求的值，以确保系统能在特殊情况下工作。指定的超时值应被视为任何给定实现的下限。真正的超时值应不晚于指定的超时值 + 50%。（这是一堆废话，按照车厂的诊断规范确定超时值）

Table 16 — Network layer timing parameter values

Timing Parameter	Description	Data Link Layer service		Timeout (ms)	Performance requirement (ms)
		Start	End		
N_As	Time for transmission of the CAN frame (any N_PDU) on the sender side	L_Data.request	L_Data.confirm	1 000	N/A
N_Ar	Time for transmission of the CAN frame (any N_PDU) on the receiver side	L_Data.request	L_Data.confirm	1 000	N/A
N_Bs	Time until reception of the next FlowControl N_PDU.	L_Data.confirm (FF), L_Data.confirm (CF), L_Data.indication (FC)	L_Data.indication (FC)	1 000	N/A
N_Br	Time until transmission of the next FlowControl N_PDU	L_Data.indication (FF), L_Data.confirm (FC)	L_Data.request (FC)	N/A	$(N_{Br} + N_{Ar}) < (0.9 * N_{Bs} \text{ timeout})$
N-Cs	Time until transmission of the next ConsecutiveFrame N_PDU	L_Data.indication (FC), L_Data.confirm (CF)	L_Data.request (CF)	N/A	$(N_{Cs} + N_{As}) < (0.9 * N_{Cr} \text{ timeout})$
N_Cr	Time until reception of the next ConsecutiveFrame N_PDU	L_Data.confirm (FC), L_Data.indication (CF)	L_Data.indication (CF)	1 000	—
s is the sender of the message. r is the receiver of the message.					

6.7.2 Network layer timeouts

Table 17 定义了网络层定时器超时产生原因和超时后的处理行为

Table 17 — Network layer timeout error handling

Timeout	Cause	Action
N_As	Any N_PDU not transmitted in time on the sender side.	Abort message transmission and issue N_USData.confirm with <N_Result> = N_TIMEOUT_A.
N_Ar	Any N_PDU not transmitted in time on the receiver side.	Abort message reception and issue N_USData.indication with <N_Result> = N_TIMEOUT_A.
N_Bs	FlowControl N_PDU not received (lost, overwritten) on the sender side or preceding FirstFrame N_PDU or ConsecutiveFrame N_PDU not received (lost, overwritten) on the receiver side.	Abort message transmission and issue N_USData.confirm with <N_Result> = N_TIMEOUT_Bs.
N_Cr	ConsecutiveFrame N_PDU not received (lost, overwritten) on the receiver side or preceding FC N_PDU not received (lost, overwritten) on the sender side.	Abort message reception and issue N_USData.indication with <N_Result> = N_TIMEOUT_Cr.

(1) N_As 和 N_Ar

N_As 和 N_Ar 可以认为是同一个 timer，是发送者本地的定时器，从网络层发出 request（网络层调用 CAN 消息发送函数）开始，到网络层收到 confirm（CAN 消息发送成功或失败）结束，如果超时就丢弃消息，并调用 N_USData.confirm 服务，with N_TIMEOUT_A。

N_Ar 超时的 Action 描述应该有误，我认为应该跟 N_As 一样，然而我并没有实现这两个 timer，这两个 timer 是为了规避本地 CAN 消息阻塞而引入的。如果系统中不会出现阻塞或者出现阻塞也不会影响到后续消息发送，则不需要实现。

(2) N_Bs

N_Bs 是发送者用来监控对端的定时器，如 Table 16 中的描述，“Time until reception of the next FlowControl N_PDU.”，N_Bs 是指到接收到下一个 FC 的最大时间，具体实现方法如下：

发送者发送完 FF 或者 BS（发送者每次连续发送的 CF 个数）个 CF 后，启动定时器 TIMER_N_BS，如果定时器超时，则应该丢弃目前正在发送的消息，并且调用 N_USData.confirm 服务，with N_TIMEOUT_Bs。如果发送者收到了 FC，则应该检查 TIMER_N_BS 定时器是否超时，如果没有超时，则关闭该定时器，继续处理 FC N_PDU，然后如果该 FC 携带的 FS = WT，则应该重新启动 TIMER_N_BS，继续等待下一个 FC；如果发现 TIMER_N_BS 已经超时，则应该丢弃该 FC。

(3) N_Br

N_Br 是接收者本地的时间参数，如 Table 16 中描述“Time until transmission of the next FlowControl N_PDU”，N_Br 是指到发送下一个 FC 的时间。后面的 Start 说明，我认为有问题，Start 中 L_Data.indication (FF) 是指接收者收到 FF（首帧），是没问题的，这一条是为了保证接收者接收到 FF 后要尽快发送 FC；Start 中的 L_Data.confirm (FC) 是指接收者成功发送 FC，这就没道理了，首先下一个 FC 的发送时间不应该参考上一个 FC，即便是参考上一个 FC，时间参数也不该跟接收到 FF 之后发送 FC 的时间参数相同。（有点绕口）。实际代码中，我没有显式的实现这一个参数，但是却满足该参数的要求，因为我收到 FF 之后没有做特殊延时，立即回复 FC，另一个情况是收到 BS 个 CF 之后，也是立即回复 FC。

(4) N-Cs

N-Cs 是发送者本地的时间参数，如 Table 16 中描述“Time until transmission of the next ConsecutiveFrame N_PDU L”，N-Cs 是指从收到 FC 或者发送完一个 CF 后，到发送下一个 CF 的时间，同样，实际代码中，我没有显式的实现这一参数，但是却满足该参数的要求，我收到 FC 之后立即发送一个 CF，而发送完成一个 CF 之后，我等待 STmin 时间后立即发送下一个 CF，并未做其他特殊延时。

(5) N_Cr

N_Cr 是接收者用来监控对端的定时器，如 Table 16 中描述“Time until reception of the next ConsecutiveFrame N_PDU”，N_Cr 是指到接收到下一个 CF 的最大时间，Table 16 中描述的 Start 是发送完 FC 或者接收到 CF，然而我在实现时稍微变通了一下，Start 变成了接收到 FF 或者 CF 后（因为收到 FF 后会立即发送 FC，收到 BS 个 CF 后也会立即发送 FC，所以不如直接在每次收到 FF 或者 CF 后启动 TIMER_N_CR），具体实现方法如下：

接收者收到一个 FF 或者 CF 之后，启动定时器 TIMER_N_CR，如果定时器超时，则应该丢弃目前正在接收的消息，并且调用 N_USData.confirm 服务 with N_TIMEOUT_Cr。如果接收者收到一个 CF，则应该检查 TIMER_N_CR 是否超时，如果没有超时，则应该关闭定时器，继续处理 CF N_PDU；如果发现 TIMER_N_CR 已经超时，则应该丢弃该 CF。

（6）STmin

STmin 没有在 Table 16 和 Table 17 中描述，但它却是协议栈需要实现的一个定时器，不同于前面几个定时参数限制最大时间，这个定时器是限制最小时间间隔的，当发送完一个 CF 后，发送者需要延时 STmin 才能发送下一帧 CF，具体实现方式如下：

发送者发送完一个 CF 之后，如果连续发送的 CF 数量没有达到 BS 个，则立即启动定时器 TIMER_STmin，在网络层主循环中运行 TIMER_STmin，当定时器超时后，立即发送下一个 CF。前面介绍流控帧时也有说明。

6.7.3 Unexpected arrival of N_PDU

6.7.3 小节介绍意外的数据单元

如果通信的一方收到了不符合正常顺序的数据单元，就叫做 unexpected N_PDU。unexpected N_PDU 可能是 SF, FF, CF, FC 或者不被本文档识别的类型。网络层的设计决定其支持全双工或者半双工通信，unexpected N_PDU 的判断跟全双工和半双工有关，两者不同。

半双工是指：同一时间，两个节点之间只允许一个方向进行通信，（A 向 B 发送 SF, FF, CF, B 向 A 回复 FC，这叫做一个方向）

全双工是指：同一时间，两个节点之间能允许两个方向进行通信，

In addition to the network layer design decision, the possibility has to be considered that a reception or transmission from/to a node with the same address information (N_AI), as contained in the received unexpected N_PDU, is in progress.

英语太渣，这句话理解不了。

普遍情况下，除了 SF 和包含物理地址的 FF，收到的其他 unexpected N_PDU 都应该被忽略；包含功能寻址的 FF 也应该被忽略。忽略的意思是指网络层不用告知上层它收到了这个 N_PDU。

Table 18 描述了网络层在收到 unexpected N_PDU 时的行为，跟网络层当前的内部状态（NWL status）和全双工/半双工有关。并且默认收到的 unexpected N_PDU 的地址信息跟正常接收和发送的地址信息一致。

Table 18 — Handling of unexpected arrival of N_PDU

NWL status	Reception of				
	SF N_PDU	FF N_PDU	CF N_PDU	FC N_PDU	Unknown N_PDU
Segmented Transmit in progress	Full-duplex: If a reception is in progress, see corresponding cell below in this table, otherwise process the SF N_PDU as the start of a new reception.	Full-duplex: If a reception is in progress, see corresponding cell below in this table, otherwise process the FF N_PDU as the start of a new reception.	Full-duplex: If a reception is in progress, see corresponding cell below in this table.	If awaited, process the FC N_PDU, otherwise ignore it.	Ignore
	Half-duplex: ignore	Half-duplex: ignore	Half-duplex: ignore		
Segmented Receive in progress	Terminate the current reception, report an N_USData.indication, with <N_Result> set to N_UNEXP_PDU, to the upper layer, and process the SF N_PDU as the start of a new reception.	Terminate the current reception, report an N_USData.indication, with <N_Result> set to N_UNEXP_PDU, to the upper layer, and process the FF N_PDU as the start of a new reception.	If awaited, process the CF N_PDU in the on-going reception and perform the required checks (e.g. SN in right order), otherwise ignore it.	Full-duplex: If a transmission is in progress, see corresponding cell above in this table.	Ignore
				Half-duplex: ignore	
Idle ^a	Process the SF N_PDU as the start of a new reception.	Process the FF N_PDU as the start of a new reception.	Ignore	Ignore	Ignore

^a Neither a segmented transmission nor segmented reception is in progress.

我开发的系统是属于半双工的，按照半双工来解读。

Idle 是指空闲状态，起始默认状态，此时如果收到 SF 或者 FF，都当作是新的接收时序的开始，收到其他的类型的 N_PDU 都应该忽略；

在接收状态下（Segmented Receive in progress），如果收到了 SF 或者 FF，都当作新的接收时序的开始，并且要调用 N_USData.indication 服务通知上层，with N_UNEXP_PDU，

在接收状态下，如果正在等待 CF 的情况下收到了 CF，则按正常 CF 处理，如果当前没有等待 CF，则应该忽略这条 CF。

在接收状态下，如果收到了 FC，半双工系统应直接忽略。

在接收状态下，如果收到不识别的 N_PDU，应直接忽略。

可以看出来，表格并未说明在接收状态下是否允许发送，我觉得发送应该有更高优先级，所以我在开发网络层的时候，任何时候都能发送，并且如果是多帧传输，立即把 NWL status 置为发送状态。所以关于发送，我的详细实现方式如下：

在任何状态下，如果上层请求发送数据，则立即进行发送，并丢弃当前正在发送的数据（但是不丢弃接收），如果请求发送是多帧传输，则把发送状态置为发送。

在发送状态下，（Segmented Transmit in progress），收到 SF, FF, CF 都应该忽略，

在发送状态下，收到 FC，如果当前正在等待 FC，则正常处理该 FC，否则忽略，

在发送状态下，如果收到不识别的 N_PDU，应直接忽略。

这一部分功能我在网络层接收数据的入口进行处理，变量 nwl_st 指示当前网络层状态。

```
extern void
network_rcv_frame (uint8_t func_addr, uint8_t frame_buf[], uint8_t frame_dlc)
{
    uint8_t err;
    uint8_t pci_type; /* protocol control information type */

    /**
     * The reception of a CAN frame with a DLC value
     * smaller than expected shall be ignored by the
     * network layer without any further action
     */
    if (frame_dlc != UDS_VALID_FRAME_LEN) return;

    if (func_addr == 0)
        g_tatype = N_TATYPE_PHYSICAL;
    else
        g_tatype = N_TATYPE_FUNCTIONAL;

    OSMutexPend(UdsMutex, 0, &err);
    pci_type = NT_GET_PCI_TYPE (frame_buf[0]);
    switch (pci_type)
    {
        case PCI_SF:
            if (nwl_st == NWL_RECV || nwl_st == NWL_IDLE)
            {
                clear_network ();
                if (nwl_st == NWL_RECV)
                    N_USData.indication (recv_buf, recv_len, N_UNEXP_PDU);
            }
        break;
    }
}
```

```

        recv_singleframe (frame_buf, frame_dlc);
    }
    break;
case PCI_FF:
    if (nwl_st == NWL_RECV || nwl_st == NWL_IDLE)
    {
        clear_network ();
        if (nwl_st == NWL_RECV)
            N_USData.indication (recv_buf, recv_len, N_UNEXP_PDU);

        if (recv_firstframe (frame_buf, frame_dlc) > 0)
            nwl_st = NWL_RECV;
        else
            nwl_st = NWL_IDLE;
    }
    break;
case PCI_CF:
    if (nwl_st == NWL_RECV && g_wait_cf == TRUE)
    {
        if (recv_consecutiveframe (frame_buf, frame_dlc) <= 0)
        {
            clear_network ();
            nwl_st = NWL_IDLE;
        }
    }
    break;
case PCI_FC:
    if (nwl_st == NWL_XMIT && g_wait_fc == TRUE)
        if (recv_flowcontrolframe (frame_buf, frame_dlc) < 0)
        {
            clear_network ();
            nwl_st = NWL_IDLE;
        }
    break;
default:
    break;
}
OSMutexPost (UdsMutex);
}

```

6.7.4 Wait frame error handling

6.7.4 小节介绍，如果接收者连续发送等待流控帧（FC N_PDU WT）到最大次数，并且仍然不能正常接收，这时候接收者应该丢弃已经收到的消息，并且调用 N_USData.indication 服务，with N_WFT_OVRN，告知上层。

这一功能我并没有实现，因为我开发的网络层不使用（FC N_PDU WT），任何情况下都能正常接收。

6.8 Interleaving of messages

6.8 小节的内容是，网络层应该有并行传输不同地址的诊断消息的能力，

这一功能我也没有实现，现状是我们的系统诊断地址都是固定的，由车厂分配，可能这一功能对于网关是有必要的，这里就不再赘述。

7 Data link layer usage

第 7 节讲的是链路层的设计，以及扩展地址情况下，数据单元的格式，我们普遍使用的都是普通地址，这一部分也不再详细介绍了。